



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

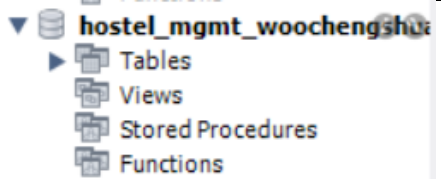
SECP3623

**Database Programming
2025/2026 SEMESTER 1**

Assignment 1 (10%)

Name	:	WOO CHENG SHUAN
Matric Id	:	A23CS0283
Section	:	02

Task 1: DATABASE CREATION AND INTEGRITY CONSTRAINTS

SQL COMMAND	OUTPUT / RESULT										
1. CREATE DATABASE											
<pre>#create database CREATE DATABASE hostel_mgmt_woochengshuan; USE hostel_mgmt_woochengshuan;</pre> Use CREATE DATABASE to create a new database.											
2. CREATE TABLES											
<pre>CREATE TABLE room_types (type_id INT AUTO_INCREMENT, type_name VARCHAR(10) NOT NULL UNIQUE, rent DECIMAL(8,2) NOT NULL, deposit DECIMAL(8,2) NOT NULL, capacity INT NOT NULL, PRIMARY KEY (type_id));</pre> Use CREATE TABLE to create table 'room_types' with the entities. Add constraints such as PRIMARY KEY, NOT NULL AND UNIQUE.	Table: room_types Columns: <table> <tr> <td>type_id</td> <td>int AI PK</td> </tr> <tr> <td>type_name</td> <td>varchar(10)</td> </tr> <tr> <td>rent</td> <td>decimal(8,2)</td> </tr> <tr> <td>deposit</td> <td>decimal(8,2)</td> </tr> <tr> <td>capacity</td> <td>int</td> </tr> </table>	type_id	int AI PK	type_name	varchar(10)	rent	decimal(8,2)	deposit	decimal(8,2)	capacity	int
type_id	int AI PK										
type_name	varchar(10)										
rent	decimal(8,2)										
deposit	decimal(8,2)										
capacity	int										
<pre>CREATE TABLE rooms (room_id INT AUTO_INCREMENT, type_id INT, room_no VARCHAR(5) NOT NULL, floor_no VARCHAR(2) NOT NULL, is_occupied BOOLEAN NOT NULL DEFAULT FALSE, PRIMARY KEY (room_id), FOREIGN KEY (type_id) REFERENCES room_types(type_id));</pre> Use CREATE TABLE to create table 'room' with the entities. Add constraints such as PRIMARY KEY, FOREIGN KEY and NOT NULL.	Table: rooms Columns: <table> <tr> <td>room_id</td> <td>int AI PK</td> </tr> <tr> <td>type_id</td> <td>int</td> </tr> <tr> <td>room_no</td> <td>varchar(5)</td> </tr> <tr> <td>floor_no</td> <td>varchar(2)</td> </tr> <tr> <td>is_occupied</td> <td>tinyint(1)</td> </tr> </table>	room_id	int AI PK	type_id	int	room_no	varchar(5)	floor_no	varchar(2)	is_occupied	tinyint(1)
room_id	int AI PK										
type_id	int										
room_no	varchar(5)										
floor_no	varchar(2)										
is_occupied	tinyint(1)										

```
CREATE TABLE students (
    student_id INT AUTO_INCREMENT,
    room_id INT,
    fname VARCHAR(40) NOT NULL,
    lname VARCHAR(40) NOT NULL,
    status ENUM ('ACTIVE', 'NON_ACTIVE'),
    checkin_date DATE,
    email VARCHAR(50) NOT NULL,
    PRIMARY KEY (student_id),
    FOREIGN KEY (room_id) REFERENCES rooms(room_id)
);
```

Use CREATE TABLE to create table 'students' with the entities. Add constraints such as PRIMARY KEY, FOREIGN KEY and NOT NULL.

Table: **students**

Columns:

student_id	int AI PK
room_id	int
fname	varchar(40)
lname	varchar(40)
status	enum('ACTIVE','NON_ACTIVE')
checkin_date	date
email	varchar(50)

```
CREATE TABLE maintenance (
    maint_id INT AUTO_INCREMENT,
    room_id INT,
    issue_desc TEXT NOT NULL,
    severity ENUM ('LOW', 'MEDIUM', 'HIGH'),
    status ENUM ('OPEN', 'RESOLVED'),
    reported_on DATE,
    resolved_on DATE,
    PRIMARY KEY (maint_id),
    FOREIGN KEY (room_id) REFERENCES rooms(room_id)
);
```

Use CREATE TABLE to create table 'maintenance' with the entities. Add constraints such as PRIMARY KEY, FOREIGN KEY and NOT NULL.

Table: **maintenance**

Columns:

maint_id	int AI PK
room_id	int
issue_desc	text
severity	enum('LOW','MEDIUM','HIGH')
status	enum('OPEN','RESOLVED')
reported_on	date
resolved_on	date

```
CREATE TABLE payments (
    payment_id INT AUTO_INCREMENT,
    student_id INT,
    amount DECIMAL(10,2) NOT NULL,
    paid_on DATE,
    method ENUM ('CASH', 'FPX', 'CARD', 'TNG'),
    note TEXT NOT NULL,
    PRIMARY KEY (payment_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id)
);
```

Use CREATE TABLE to create table 'payments' with the entities. Add constraints such as PRIMARY KEY, FOREIGN KEY and NOT NULL.

Table: **payments**

Columns:

payment_id	int AI PK
student_id	int
amount	decimal(10,2)
paid_on	date
method	enum('CASH','FPX','CARD','TNG')
note	text

3. <i>ALTER TABLE: add an email column to students and mark it UNIQUE</i>															
<pre>ALTER TABLE students MODIFY email VARCHAR(50) NOT NULL UNIQUE;</pre> Use ALTER TABLE to MODIFY email columns into UNIQUE.	Table: students Columns: <table> <tr><td>student_id</td><td>int AI PK</td></tr> <tr><td>room_id</td><td>int</td></tr> <tr><td>fname</td><td>varchar(40)</td></tr> <tr><td>lname</td><td>varchar(40)</td></tr> <tr><td>status</td><td>enum('ACTIVE','NON_ACTIVE')</td></tr> <tr><td>checkin_date</td><td>date</td></tr> <tr><td>email</td><td>varchar(50)</td></tr> </table>	student_id	int AI PK	room_id	int	fname	varchar(40)	lname	varchar(40)	status	enum('ACTIVE','NON_ACTIVE')	checkin_date	date	email	varchar(50)
student_id	int AI PK														
room_id	int														
fname	varchar(40)														
lname	varchar(40)														
status	enum('ACTIVE','NON_ACTIVE')														
checkin_date	date														
email	varchar(50)														
4. <i>DROP TABLE: create a temporary test table and drop it safely</i>															
<pre>CREATE TABLE temp_test (test_id INT); DROP TABLE temp_test;</pre>	✔ 10 01:11:59 CREATE TABLE temp_test (test_id INT) ✔ 11 01:12:05 DROP TABLE temp_test														

TASK 2: DATA MANIPULATION AND FILTERING

SQL COMMAND	OUTPUT / RESULT																																																																	
1. DATA INSERTION (DML)																																																																		
<pre>INSERT INTO room_types (type_name, rent, deposit, capacity) VALUES ('Single', '350.00', '105.00', '1'), ('Double', '500.00', '150.00', '2'), ('Premium', '900.00', '270.00', '2'), ('Family', '1200.00', '360.00', '4'), ('Economy', '420.00', '126.00', '1'), ('Studio', '700.00', '210.00', '1'), ('Deluxe', '650.00', '195.00', '2'), ('Suite', '1500.00', '450.00', '3'), ('Accessible', '480.00', '144.00', '2'), ('Shared', '300.00', '90.00', '3');</pre> Use INSERT INTO ... VALUES to insert 10 valid records into the table 'room_types' while the 'type_id' column data is created by auto increment. Ensuring that there is a containing value between 400 and 800.	<table><tr><th>type_id</th><th>type_name</th><th>rent</th><th>deposit</th><th>capacity</th></tr><tr><td>1</td><td>Single</td><td>350.00</td><td>105.00</td><td>1</td></tr><tr><td>2</td><td>Double</td><td>500.00</td><td>150.00</td><td>2</td></tr><tr><td>3</td><td>Premium</td><td>900.00</td><td>270.00</td><td>2</td></tr><tr><td>4</td><td>Family</td><td>1200.00</td><td>360.00</td><td>4</td></tr><tr><td>5</td><td>Economy</td><td>420.00</td><td>126.00</td><td>1</td></tr><tr><td>6</td><td>Studio</td><td>700.00</td><td>210.00</td><td>1</td></tr><tr><td>7</td><td>Deluxe</td><td>650.00</td><td>195.00</td><td>2</td></tr><tr><td>8</td><td>Suite</td><td>1500.00</td><td>450.00</td><td>3</td></tr><tr><td>9</td><td>Accessible</td><td>480.00</td><td>144.00</td><td>2</td></tr><tr><td>10</td><td>Shared</td><td>300.00</td><td>90.00</td><td>3</td></tr></table>	type_id	type_name	rent	deposit	capacity	1	Single	350.00	105.00	1	2	Double	500.00	150.00	2	3	Premium	900.00	270.00	2	4	Family	1200.00	360.00	4	5	Economy	420.00	126.00	1	6	Studio	700.00	210.00	1	7	Deluxe	650.00	195.00	2	8	Suite	1500.00	450.00	3	9	Accessible	480.00	144.00	2	10	Shared	300.00	90.00	3										
type_id	type_name	rent	deposit	capacity																																																														
1	Single	350.00	105.00	1																																																														
2	Double	500.00	150.00	2																																																														
3	Premium	900.00	270.00	2																																																														
4	Family	1200.00	360.00	4																																																														
5	Economy	420.00	126.00	1																																																														
6	Studio	700.00	210.00	1																																																														
7	Deluxe	650.00	195.00	2																																																														
8	Suite	1500.00	450.00	3																																																														
9	Accessible	480.00	144.00	2																																																														
10	Shared	300.00	90.00	3																																																														
<pre>INSERT INTO rooms (type_id, room_no, floor_no) VALUES (1, 'A101', 1), (2, 'A102', 1), (3, 'A103', 1), (4, 'B101', 2), (5, 'B102', 2), (6, 'B103', 2), (7, 'C101', 3), (8, 'C102', 3), (9, 'C103', 3), (10, 'C104', 3);</pre> Use INSERT INTO ... VALUES to insert 10 valid records into the table 'rooms' while the 'room_id' column data is created by auto increment and the 'is_occupied' is created by a default value of FALSE (0).	<table><tr><th>room_id</th><th>type_id</th><th>room_no</th><th>floor_no</th><th>is_occupied</th></tr><tr><td>1</td><td>1</td><td>A101</td><td>1</td><td>1</td></tr><tr><td>2</td><td>2</td><td>A102</td><td>1</td><td>1</td></tr><tr><td>3</td><td>3</td><td>A103</td><td>1</td><td>1</td></tr><tr><td>4</td><td>4</td><td>B101</td><td>2</td><td>1</td></tr><tr><td>5</td><td>5</td><td>B102</td><td>2</td><td>1</td></tr><tr><td>6</td><td>6</td><td>B103</td><td>2</td><td>0</td></tr><tr><td>7</td><td>7</td><td>C101</td><td>3</td><td>1</td></tr><tr><td>8</td><td>8</td><td>C102</td><td>3</td><td>0</td></tr><tr><td>9</td><td>9</td><td>C103</td><td>3</td><td>0</td></tr><tr><td>10</td><td>10</td><td>C104</td><td>3</td><td>0</td></tr><tr><td>15</td><td>1</td><td>A104</td><td>1</td><td>0</td></tr><tr><td>16</td><td>1</td><td>A105</td><td>1</td><td>0</td></tr></table>	room_id	type_id	room_no	floor_no	is_occupied	1	1	A101	1	1	2	2	A102	1	1	3	3	A103	1	1	4	4	B101	2	1	5	5	B102	2	1	6	6	B103	2	0	7	7	C101	3	1	8	8	C102	3	0	9	9	C103	3	0	10	10	C104	3	0	15	1	A104	1	0	16	1	A105	1	0
room_id	type_id	room_no	floor_no	is_occupied																																																														
1	1	A101	1	1																																																														
2	2	A102	1	1																																																														
3	3	A103	1	1																																																														
4	4	B101	2	1																																																														
5	5	B102	2	1																																																														
6	6	B103	2	0																																																														
7	7	C101	3	1																																																														
8	8	C102	3	0																																																														
9	9	C103	3	0																																																														
10	10	C104	3	0																																																														
15	1	A104	1	0																																																														
16	1	A105	1	0																																																														
<pre>INSERT INTO rooms (type_id, room_no, floor_no) VALUES (1, 'A104', 1), (1, 'A105', 1);</pre> Data is added for the second time since I noticed there is no output value for the fourth question of the summary queries part.																																																																		

```

INSERT INTO students (room_id, fname, lname, status, checkin_date, email)
VALUES
(1, 'Liam', 'Ong', 'ACTIVE', '2024-09-01', 'liamong@gmail.com'),
(NULL, 'Noah', 'Lee', 'NON_ACTIVE', NULL, 'noahlee@gmail.com'),
(2, 'Oliver', 'Zhang', 'ACTIVE', '2024-10-01', 'oliverzhang@gmail.com'),
(3, 'Emma', 'Low', 'ACTIVE', '2024-09-20', 'emmalow@gmail.com'),
(NULL, 'Grace', 'Chen', 'NON_ACTIVE', NULL, 'gracechen@gmail.com'),
(4, 'Amelia', 'Woo', 'ACTIVE', '2024-10-12', 'ameliawoo@gmail.com'),
(5, 'Charlotte', 'Heng', 'ACTIVE', '2024-10-18', 'charlotteheng@gmail.com'),
(NULL, 'Mia', 'Cheong', 'NON_ACTIVE', NULL, 'miacheong@gmail.com'),
(7, 'Jackson', 'Wang', 'ACTIVE', '2024-08-22', 'jacksonwang@gmail.com'),
(NULL, 'Henry', 'Chau', 'NON_ACTIVE', NULL, 'henrychau@gmail.com');

```

Use INSERT INTO ... VALUES to insert 10 valid records into the table 'students' while the 'student_id' column data is created by auto increment. Ensuring that there is at least one student's name starts with the letter 'A'.

```

INSERT INTO students (room_id, fname, lname, status, checkin_date, email)
VALUES
(15, 'Vivian', 'Chang', 'ACTIVE', '2024-09-02', 'vivianchang@gmail.com'),
(16, 'Daniel', 'Kang', 'ACTIVE', '2024-09-03', 'danielkang@gmail.com');

```

Data is added for the second time since I noticed there is no output value for the fourth question of the summary queries part.

student_id	room_id	fname	lname	status	checkin_date	email
1	1	Liam	Ong	ACTIVE	2024-09-01	liamong@gmail.com
2	2	Noah	Lee	NON_ACTIVE	NULL	noahlee@gmail.com
3	2	Oliver	Zhang	ACTIVE	2024-10-01	oliverzhang@gmail.com
4	3	Emma	Low	ACTIVE	2024-09-20	emmalow@gmail.com
5	3	Grace	Chen	NON_ACTIVE	NULL	gracechen@gmail.com
6	4	Amelia	Woo	ACTIVE	2024-10-12	ameliawoo@gmail.com
7	5	Charlotte	Heng	ACTIVE	2024-10-18	charlotteheng@gmail.com
8	5	Mia	Cheong	NON_ACTIVE	NULL	miacheong@gmail.com
9	7	Jackson	Wang	ACTIVE	2024-08-22	jacksonwang@gmail.com
10	7	Henry	Chau	NON_ACTIVE	NULL	henrychau@gmail.com
13	15	Vivian	Chang	ACTIVE	2024-09-02	vivianchang@gmail.com
14	16	Daniel	Kang	ACTIVE	2024-09-03	danielkang@gmail.com

```

INSERT INTO maintenance (room_id, issue_desc, severity, status, reported_on, resolved_on)
VALUES
(1, 'WATER LEAKAGE', 'HIGH', 'RESOLVED', '2024-08-23', '2024-08-24'),
(2, 'NO WATER', 'HIGH', 'OPEN', '2024-09-02', NULL),
(3, 'NO ELECTRIC', 'MEDIUM', 'RESOLVED', '2024-06-14', '2024-06-15'),
(4, 'BROKEN WINDOW', 'LOW', 'OPEN', '2024-09-12', NULL),
(5, 'AC NOT COOLING', 'MEDIUM', 'RESOLVED', '2024-07-26', '2024-07-28'),
(6, 'CEILING CRACK', 'LOW', 'OPEN', '2024-09-20', NULL),
(7, 'BROKEN BED FRAME', 'MEDIUM', 'OPEN', '2024-09-15', NULL),
(8, 'BAD SMELL', 'HIGH', 'RESOLVED', '2024-07-30', '2024-08-05'),
(9, 'DOOR CANNOT LOCK', 'HIGH', 'RESOLVED', '2024-06-03', '2024-06-05'),
(10, 'SUITS SPOILED', 'MEDIUM', 'RESOLVED', '2024-06-27', '2024-06-30');

```

Use INSERT INTO ... VALUES to insert 10 valid records into the table 'maintenance' while the 'maint_id' column data is created by auto increment, including three severity types and two statuses in the table.

```

INSERT INTO maintenance (room_id, issue_desc, severity, status, reported_on, resolved_on)
VALUES
(15, 'NO WATER', 'HIGH', 'OPEN', '2024-09-03', NULL),
(16, 'NO ELECTRIC', 'MEDIUM', 'OPEN', '2024-09-04', NULL);

```

Data is added for the second time since I noticed there is no output value for the fourth question of the summary queries part.

maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on
1	1	WATER LEAKAGE	HIGH	RESOLVED	2024-08-23	2024-08-24
2	2	NO WATER	HIGH	OPEN	2024-09-02	NULL
3	3	NO ELECTRIC	MEDIUM	RESOLVED	2024-06-14	2024-06-15
4	4	BROKEN WINDOW	LOW	OPEN	2024-09-12	NULL
5	5	AC NOT COOLING	MEDIUM	RESOLVED	2024-07-26	2024-07-28
6	6	CEILING CRACK	LOW	OPEN	2024-09-20	NULL
7	7	BROKEN BED FRAME	MEDIUM	OPEN	2024-09-15	NULL
8	8	BAD SMELL	HIGH	RESOLVED	2024-07-30	2024-08-05
9	9	DOOR CANNOT LOCK	HIGH	RESOLVED	2024-06-03	2024-06-05
10	10	SUITS SPOILED	MEDIUM	RESOLVED	2024-06-27	2024-06-30

maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on
2	2	NO WATER	HIGH	OPEN	2024-09-02	NULL
4	4	BROKEN WINDOW	LOW	OPEN	2024-09-12	NULL
6	6	CEILING CRACK	LOW	OPEN	2024-09-20	NULL
7	7	BROKEN BED FRAME	MEDIUM	OPEN	2024-09-15	NULL
13	15	NO WATER	HIGH	OPEN	2024-09-03	NULL
14	16	NO ELECTRIC	MEDIUM	OPEN	2024-09-04	NULL

<pre>INSERT INTO payments (student_id, amount, paid_on, method, note) VALUES (1, 350.00, '2024-06-01', 'TNG', 'JUNE RENT'), (2, 500.00, '2024-06-03', 'CASH', 'JUNE RENT'), (3, 420.00, '2024-07-01', 'TNG', 'JULY RENT'), (4, 700.00, '2024-07-08', 'CARD', 'JULY RENT'), (5, 900.00, '2024-08-02', 'FPX', 'AUGUST RENT'), (6, 300.00, '2024-08-10', 'CASH', 'AUGUST RENT'), (7, 650.00, '2024-09-05', 'TNG', 'SEPTEMBER RENT'), (8, 300.00, '2024-09-07', 'CARD', 'SEPTEMBER RENT'), (9, 480.00, '2024-10-02', 'FPX', 'OCTOBER RENT'), (10, 420.00, '2024-10-09', 'TNG', 'OCTOBER RENT');</pre>	<table><tr><th>payment_id</th><th>student_id</th><th>amount</th><th>paid_on</th><th>method</th><th>note</th></tr><tr><td>1</td><td>1</td><td>350.00</td><td>2024-06-01</td><td>TNG</td><td>JUNE RENT</td></tr><tr><td>2</td><td>2</td><td>500.00</td><td>2024-06-03</td><td>CASH</td><td>JUNE RENT</td></tr><tr><td>3</td><td>3</td><td>420.00</td><td>2024-07-01</td><td>TNG</td><td>JULY RENT</td></tr><tr><td>4</td><td>4</td><td>700.00</td><td>2024-07-08</td><td>CARD</td><td>JULY RENT</td></tr><tr><td>5</td><td>5</td><td>900.00</td><td>2024-08-02</td><td>FPX</td><td>AUGUST RENT</td></tr><tr><td>6</td><td>6</td><td>300.00</td><td>2024-08-10</td><td>CASH</td><td>AUGUST RENT</td></tr><tr><td>7</td><td>7</td><td>650.00</td><td>2024-09-05</td><td>TNG</td><td>SEPTEMBER RENT</td></tr><tr><td>8</td><td>8</td><td>300.00</td><td>2024-09-07</td><td>CARD</td><td>SEPTEMBER RENT</td></tr><tr><td>9</td><td>9</td><td>480.00</td><td>2024-10-02</td><td>FPX</td><td>OCTOBER RENT</td></tr><tr><td>10</td><td>10</td><td>420.00</td><td>2024-10-09</td><td>TNG</td><td>OCTOBER RENT</td></tr></table>	payment_id	student_id	amount	paid_on	method	note	1	1	350.00	2024-06-01	TNG	JUNE RENT	2	2	500.00	2024-06-03	CASH	JUNE RENT	3	3	420.00	2024-07-01	TNG	JULY RENT	4	4	700.00	2024-07-08	CARD	JULY RENT	5	5	900.00	2024-08-02	FPX	AUGUST RENT	6	6	300.00	2024-08-10	CASH	AUGUST RENT	7	7	650.00	2024-09-05	TNG	SEPTEMBER RENT	8	8	300.00	2024-09-07	CARD	SEPTEMBER RENT	9	9	480.00	2024-10-02	FPX	OCTOBER RENT	10	10	420.00	2024-10-09	TNG	OCTOBER RENT
payment_id	student_id	amount	paid_on	method	note																																																														
1	1	350.00	2024-06-01	TNG	JUNE RENT																																																														
2	2	500.00	2024-06-03	CASH	JUNE RENT																																																														
3	3	420.00	2024-07-01	TNG	JULY RENT																																																														
4	4	700.00	2024-07-08	CARD	JULY RENT																																																														
5	5	900.00	2024-08-02	FPX	AUGUST RENT																																																														
6	6	300.00	2024-08-10	CASH	AUGUST RENT																																																														
7	7	650.00	2024-09-05	TNG	SEPTEMBER RENT																																																														
8	8	300.00	2024-09-07	CARD	SEPTEMBER RENT																																																														
9	9	480.00	2024-10-02	FPX	OCTOBER RENT																																																														
10	10	420.00	2024-10-09	TNG	OCTOBER RENT																																																														
Use INSERT INTO ... VALUES to insert 10 valid records into the table ‘payments’ while the ‘payment_id’ column data is created by auto increment, including four payment methods in the table.																																																																			

2. UPDATE AND DELETE OPERATIONS

<pre>UPDATE rooms r JOIN (SELECT room_id FROM students WHERE status = 'ACTIVE' AND room_id IS NOT NULL GROUP BY room_id) s ON r.room_id = s.room_id SET r.is_occupied = 1;</pre>	<table><tr><th>room_id</th><th>type_id</th><th>room_no</th><th>floor_no</th><th>is_occupied</th></tr><tr><td>1</td><td>1</td><td>A101</td><td>1</td><td>1</td></tr><tr><td>2</td><td>2</td><td>A102</td><td>1</td><td>1</td></tr><tr><td>3</td><td>3</td><td>A103</td><td>1</td><td>1</td></tr><tr><td>4</td><td>4</td><td>B101</td><td>2</td><td>1</td></tr><tr><td>5</td><td>5</td><td>B102</td><td>2</td><td>1</td></tr><tr><td>6</td><td>6</td><td>B103</td><td>2</td><td>0</td></tr><tr><td>7</td><td>7</td><td>C101</td><td>3</td><td>1</td></tr><tr><td>8</td><td>8</td><td>C102</td><td>3</td><td>0</td></tr><tr><td>9</td><td>9</td><td>C103</td><td>3</td><td>0</td></tr><tr><td>10</td><td>10</td><td>C104</td><td>3</td><td>0</td></tr></table>	room_id	type_id	room_no	floor_no	is_occupied	1	1	A101	1	1	2	2	A102	1	1	3	3	A103	1	1	4	4	B101	2	1	5	5	B102	2	1	6	6	B103	2	0	7	7	C101	3	1	8	8	C102	3	0	9	9	C103	3	0	10	10	C104	3	0
room_id	type_id	room_no	floor_no	is_occupied																																																				
1	1	A101	1	1																																																				
2	2	A102	1	1																																																				
3	3	A103	1	1																																																				
4	4	B101	2	1																																																				
5	5	B102	2	1																																																				
6	6	B103	2	0																																																				
7	7	C101	3	1																																																				
8	8	C102	3	0																																																				
9	9	C103	3	0																																																				
10	10	C104	3	0																																																				
Update the column ‘is_occupied’ to (1) from the table of ‘room’ when it finds out that there is at least one student ‘ACTIVE’ in the same ‘room_id’ from the table of ‘students’ and ‘rooms’.																																																								

<pre>DELETE FROM maintenance WHERE status = 'RESOLVED' AND reported_on < DATE_SUB(curdate(), INTERVAL 60 DAY);</pre>	<table><tr><th>maint_id</th><th>room_id</th><th>issue_desc</th><th>severity</th><th>status</th><th>reported_on</th><th>resolved_on</th></tr><tr><td>2</td><td>2</td><td>NO WATER</td><td>HIGH</td><td>OPEN</td><td>2024-09-02</td><td>DATE</td></tr><tr><td>4</td><td>4</td><td>BROKEN WINDOW</td><td>LOW</td><td>OPEN</td><td>2024-09-12</td><td>DATE</td></tr><tr><td>6</td><td>6</td><td>CEILING CRACK</td><td>LOW</td><td>OPEN</td><td>2024-09-20</td><td>DATE</td></tr><tr><td>7</td><td>7</td><td>BROKEN BED FRAME</td><td>MEDIUM</td><td>OPEN</td><td>2024-09-15</td><td>DATE</td></tr></table>	maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on	2	2	NO WATER	HIGH	OPEN	2024-09-02	DATE	4	4	BROKEN WINDOW	LOW	OPEN	2024-09-12	DATE	6	6	CEILING CRACK	LOW	OPEN	2024-09-20	DATE	7	7	BROKEN BED FRAME	MEDIUM	OPEN	2024-09-15	DATE
maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on																														
2	2	NO WATER	HIGH	OPEN	2024-09-02	DATE																														
4	4	BROKEN WINDOW	LOW	OPEN	2024-09-12	DATE																														
6	6	CEILING CRACK	LOW	OPEN	2024-09-20	DATE																														
7	7	BROKEN BED FRAME	MEDIUM	OPEN	2024-09-15	DATE																														
Delete the row in the table ‘maintenance’ when the status shows ‘RESOLVED’ and has already passed 60 days.																																				

3. DATA RETRIEVAL AND FILTERING QUERIES

<pre>SELECT * FROM room_types WHERE rent BETWEEN 400 AND 800;</pre>	<table><tr><th>type_id</th><th>type_name</th><th>rent</th><th>deposit</th><th>capacity</th></tr><tr><td>2</td><td>Double</td><td>500.00</td><td>150.00</td><td>2</td></tr><tr><td>5</td><td>Economy</td><td>420.00</td><td>126.00</td><td>1</td></tr><tr><td>6</td><td>Studio</td><td>700.00</td><td>210.00</td><td>1</td></tr><tr><td>7</td><td>Deluxe</td><td>650.00</td><td>195.00</td><td>2</td></tr><tr><td>9</td><td>Accessible</td><td>480.00</td><td>144.00</td><td>2</td></tr></table>	type_id	type_name	rent	deposit	capacity	2	Double	500.00	150.00	2	5	Economy	420.00	126.00	1	6	Studio	700.00	210.00	1	7	Deluxe	650.00	195.00	2	9	Accessible	480.00	144.00	2
type_id	type_name	rent	deposit	capacity																											
2	Double	500.00	150.00	2																											
5	Economy	420.00	126.00	1																											
6	Studio	700.00	210.00	1																											
7	Deluxe	650.00	195.00	2																											
9	Accessible	480.00	144.00	2																											
Display the row from the table ‘room_types’ where the rent value is between RM400 and RM800.																															

<pre>SELECT * FROM students WHERE fname LIKE 'A%';</pre>	<table><tr><th>student_id</th><th>room_id</th><th>fname</th><th>lname</th><th>status</th><th>checkin_date</th><th>email</th></tr><tr><td>6</td><td>4</td><td>Amelia</td><td>Woo</td><td>ACTIVE</td><td>2024-10-12</td><td>ameliawoo@gmail.com</td></tr></table>	student_id	room_id	fname	lname	status	checkin_date	email	6	4	Amelia	Woo	ACTIVE	2024-10-12	ameliawoo@gmail.com
student_id	room_id	fname	lname	status	checkin_date	email									
6	4	Amelia	Woo	ACTIVE	2024-10-12	ameliawoo@gmail.com									

Display the row from the table 'students' where the first name of the student starts with the letter 'A'.																																				
<pre>SELECT * FROM payments WHERE method IN ('FPX', 'CARD');</pre> Display the row from the table 'payments' where the students pay using 'FPX' or 'CARD'.	<table><tr><th>payment_id</th><th>student_id</th><th>amount</th><th>paid_on</th><th>method</th><th>note</th></tr><tr><td>4</td><td>4</td><td>700.00</td><td>2024-07-08</td><td>CARD</td><td>JULY RENT</td></tr><tr><td>5</td><td>5</td><td>900.00</td><td>2024-08-02</td><td>FPX</td><td>AUGUST RENT</td></tr><tr><td>8</td><td>8</td><td>300.00</td><td>2024-09-07</td><td>CARD</td><td>SEPTEMBER RENT</td></tr><tr><td>9</td><td>9</td><td>480.00</td><td>2024-10-02</td><td>FPX</td><td>OCTOBER RENT</td></tr></table>	payment_id	student_id	amount	paid_on	method	note	4	4	700.00	2024-07-08	CARD	JULY RENT	5	5	900.00	2024-08-02	FPX	AUGUST RENT	8	8	300.00	2024-09-07	CARD	SEPTEMBER RENT	9	9	480.00	2024-10-02	FPX	OCTOBER RENT					
payment_id	student_id	amount	paid_on	method	note																															
4	4	700.00	2024-07-08	CARD	JULY RENT																															
5	5	900.00	2024-08-02	FPX	AUGUST RENT																															
8	8	300.00	2024-09-07	CARD	SEPTEMBER RENT																															
9	9	480.00	2024-10-02	FPX	OCTOBER RENT																															
<pre>SELECT * FROM maintenance WHERE (severity = 'HIGH' AND STATUS = 'OPEN') OR (NOT (severity IN ('LOW')) AND STATUS = 'OPEN'));</pre> Display the row from the table 'maintenance' where the severity is high and medium, and status with 'OPEN'.	<table><tr><th>maint_id</th><th>room_id</th><th>issue_desc</th><th>severity</th><th>status</th><th>reported_on</th><th>resolved_on</th></tr><tr><td>2</td><td>2</td><td>NO WATER</td><td>HIGH</td><td>OPEN</td><td>2024-09-02</td><td>NULL</td></tr><tr><td>7</td><td>7</td><td>BROKEN BED FRAME</td><td>MEDIUM</td><td>OPEN</td><td>2024-09-15</td><td>NULL</td></tr><tr><td>13</td><td>15</td><td>NO WATER</td><td>HIGH</td><td>OPEN</td><td>2024-09-03</td><td>NULL</td></tr><tr><td>14</td><td>16</td><td>NO ELECTRIC</td><td>MEDIUM</td><td>OPEN</td><td>2024-09-04</td><td>NULL</td></tr></table>	maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on	2	2	NO WATER	HIGH	OPEN	2024-09-02	NULL	7	7	BROKEN BED FRAME	MEDIUM	OPEN	2024-09-15	NULL	13	15	NO WATER	HIGH	OPEN	2024-09-03	NULL	14	16	NO ELECTRIC	MEDIUM	OPEN	2024-09-04	NULL
maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on																														
2	2	NO WATER	HIGH	OPEN	2024-09-02	NULL																														
7	7	BROKEN BED FRAME	MEDIUM	OPEN	2024-09-15	NULL																														
13	15	NO WATER	HIGH	OPEN	2024-09-03	NULL																														
14	16	NO ELECTRIC	MEDIUM	OPEN	2024-09-04	NULL																														
4. FUNCTION AND EXPRESSIONS																																				
<pre>SELECT method, SUM(amount) AS total_amount FROM payments WHERE method IN ('FPX', 'CARD', 'TNG', 'CASH') GROUP BY method;</pre> Calculate the sum of the amount with the category of payment method and display it in a table.	<table><tr><th>method</th><th>total_amount</th></tr><tr><td>TNG</td><td>1840.00</td></tr><tr><td>CASH</td><td>800.00</td></tr><tr><td>CARD</td><td>1000.00</td></tr><tr><td>FPX</td><td>1380.00</td></tr></table>	method	total_amount	TNG	1840.00	CASH	800.00	CARD	1000.00	FPX	1380.00																									
method	total_amount																																			
TNG	1840.00																																			
CASH	800.00																																			
CARD	1000.00																																			
FPX	1380.00																																			
<pre>SELECT student_id, UPPER(CONCAT(fname, ' ', lname)) AS full_name, LENGTH(CONCAT(fname, ' ', lname)) AS full_name_length FROM students;</pre> Display the combined first name and last name as 'full_name' and mention the total length of the first name and last name in a table.	<table><tr><th>student_id</th><th>full_name</th><th>full_name_length</th></tr><tr><td>1</td><td>LIAM ONG</td><td>8</td></tr><tr><td>2</td><td>NOAH LEE</td><td>8</td></tr><tr><td>3</td><td>OLIVER ZHANG</td><td>12</td></tr><tr><td>4</td><td>EMMA LOW</td><td>8</td></tr><tr><td>5</td><td>GRACE CHEN</td><td>10</td></tr><tr><td>6</td><td>AMELIA WOO</td><td>10</td></tr><tr><td>7</td><td>CHARLOTTE HENG</td><td>14</td></tr><tr><td>8</td><td>MIA CHEONG</td><td>10</td></tr><tr><td>9</td><td>JACKSON WANG</td><td>12</td></tr><tr><td>10</td><td>HENRY CHAU</td><td>10</td></tr></table>	student_id	full_name	full_name_length	1	LIAM ONG	8	2	NOAH LEE	8	3	OLIVER ZHANG	12	4	EMMA LOW	8	5	GRACE CHEN	10	6	AMELIA WOO	10	7	CHARLOTTE HENG	14	8	MIA CHEONG	10	9	JACKSON WANG	12	10	HENRY CHAU	10		
student_id	full_name	full_name_length																																		
1	LIAM ONG	8																																		
2	NOAH LEE	8																																		
3	OLIVER ZHANG	12																																		
4	EMMA LOW	8																																		
5	GRACE CHEN	10																																		
6	AMELIA WOO	10																																		
7	CHARLOTTE HENG	14																																		
8	MIA CHEONG	10																																		
9	JACKSON WANG	12																																		
10	HENRY CHAU	10																																		

TASK 3: REPORTING AND AGGREGATION

SQL COMMAND	OUTPUT / RESULT																																																																																								
1. CREATE A VIEW v_room_status WITH COLUMNS																																																																																									
<pre>CREATE VIEW v_room_status AS SELECT r.room_no, rt.type_name, rt.rent, r.floor_no, rt.capacity, COUNT(DISTINCT s.student_id) AS n_occupants, COUNT(DISTINCT m.maint_id) AS pending_issues, CASE WHEN COUNT(DISTINCT s.student_id) = 0 THEN TRUE ELSE FALSE END AS is_vacant FROM rooms r JOIN room_types rt ON r.type_id = rt.type_id LEFT JOIN students s ON s.room_id = r.room_id AND s.status = 'ACTIVE' LEFT JOIN maintenance m ON m.room_id = r.room_id AND s.status = 'OPEN' GROUP BY r.room_no, rt.type_name, rt.rent, r.floor_no, rt.capacity;</pre> USE CREATE VIEW to create a virtual table named 'v_room_status'. Use SELECT the column of 'room_no', 'type_name', 'floor_no', 'capacity' directly from the original tables, which are 'rooms' stands for r, and 'room_types' stands for rt. Use COUNT to count the number of students checked into the room and name as 'n_occupants'. Also, count the number of pending maintenance tasks for the room and name it as 'pending_issues'. Use CASE WHEN to identify whether the room is vacant or not in a boolean state. These lines JOIN each room with its room type, active students, and open maintenance issues, then GROUP the data so every room appears as one row with correct counts of occupants and issues.	<table><tr><th>room_no</th><th>type_name</th><th>rent</th><th>floor_no</th><th>capacity</th><th>n_occupants</th><th>pending_issues</th><th>is_vacant</th></tr><tr><td>A101</td><td>Single</td><td>350.00</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>A102</td><td>Double</td><td>500.00</td><td>1</td><td>2</td><td>1</td><td>0</td><td>0</td></tr><tr><td>A103</td><td>Premium</td><td>900.00</td><td>1</td><td>2</td><td>1</td><td>0</td><td>0</td></tr><tr><td>B101</td><td>Family</td><td>1200.00</td><td>2</td><td>4</td><td>1</td><td>0</td><td>0</td></tr><tr><td>B102</td><td>Economy</td><td>420.00</td><td>2</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>B103</td><td>Studio</td><td>700.00</td><td>2</td><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>C101</td><td>Deluxe</td><td>650.00</td><td>3</td><td>2</td><td>1</td><td>0</td><td>0</td></tr><tr><td>C102</td><td>Suite</td><td>1500.00</td><td>3</td><td>3</td><td>0</td><td>0</td><td>1</td></tr><tr><td>C103</td><td>Accessible</td><td>480.00</td><td>3</td><td>2</td><td>0</td><td>0</td><td>1</td></tr><tr><td>C104</td><td>Shared</td><td>300.00</td><td>3</td><td>3</td><td>0</td><td>0</td><td>1</td></tr></table>	room_no	type_name	rent	floor_no	capacity	n_occupants	pending_issues	is_vacant	A101	Single	350.00	1	1	1	0	0	A102	Double	500.00	1	2	1	0	0	A103	Premium	900.00	1	2	1	0	0	B101	Family	1200.00	2	4	1	0	0	B102	Economy	420.00	2	1	1	0	0	B103	Studio	700.00	2	1	0	0	1	C101	Deluxe	650.00	3	2	1	0	0	C102	Suite	1500.00	3	3	0	0	1	C103	Accessible	480.00	3	2	0	0	1	C104	Shared	300.00	3	3	0	0	1
room_no	type_name	rent	floor_no	capacity	n_occupants	pending_issues	is_vacant																																																																																		
A101	Single	350.00	1	1	1	0	0																																																																																		
A102	Double	500.00	1	2	1	0	0																																																																																		
A103	Premium	900.00	1	2	1	0	0																																																																																		
B101	Family	1200.00	2	4	1	0	0																																																																																		
B102	Economy	420.00	2	1	1	0	0																																																																																		
B103	Studio	700.00	2	1	0	0	1																																																																																		
C101	Deluxe	650.00	3	2	1	0	0																																																																																		
C102	Suite	1500.00	3	3	0	0	1																																																																																		
C103	Accessible	480.00	3	2	0	0	1																																																																																		
C104	Shared	300.00	3	3	0	0	1																																																																																		
2. SUMMARY QUERIES																																																																																									
i. Total number of students per room type																																																																																									
<pre>SELECT rt.type_name, COUNT(s.student_id) AS total_students FROM room_types rt LEFT JOIN rooms r ON r.type_id = rt.type_id LEFT JOIN students s ON s.room_id = r.room_id GROUP BY rt.type_name;</pre> Use COUNT to calculate the total number of students for each room type. The query starts from 'room_types', LEFT JOIN it with 'rooms' and then 'students' to include all room types even if they have	<table><tr><th>type_name</th><th>total_students</th></tr><tr><td>Accessible</td><td>0</td></tr><tr><td>Deluxe</td><td>1</td></tr><tr><td>Double</td><td>1</td></tr><tr><td>Economy</td><td>1</td></tr><tr><td>Family</td><td>1</td></tr><tr><td>Premium</td><td>1</td></tr><tr><td>Shared</td><td>0</td></tr><tr><td>Single</td><td>3</td></tr><tr><td>Studio</td><td>0</td></tr><tr><td>Suite</td><td>0</td></tr></table>	type_name	total_students	Accessible	0	Deluxe	1	Double	1	Economy	1	Family	1	Premium	1	Shared	0	Single	3	Studio	0	Suite	0																																																																		
type_name	total_students																																																																																								
Accessible	0																																																																																								
Deluxe	1																																																																																								
Double	1																																																																																								
Economy	1																																																																																								
Family	1																																																																																								
Premium	1																																																																																								
Shared	0																																																																																								
Single	3																																																																																								
Studio	0																																																																																								
Suite	0																																																																																								

no rooms or students, and finally GROUP BY the room type.																							
ii. Average rent and total deposit per room type																							
<pre>SELECT ROUND(AVG(rt.rent), 2) AS avg_rent, ROUND(SUM(rt.deposit), 2) AS total_deposit FROM room_types rt GROUP BY rt.type_name;</pre> ROUND up the average rent to 2 decimal points and ROUND the SUM of the total deposit to 2 decimal points. GROUP them to make sure the calculation is done for each room type.	<table><tr><th>avg_rent</th><th>total_deposit</th></tr><tr><td>480.00</td><td>144.00</td></tr><tr><td>650.00</td><td>195.00</td></tr><tr><td>500.00</td><td>150.00</td></tr><tr><td>420.00</td><td>126.00</td></tr><tr><td>1200.00</td><td>360.00</td></tr><tr><td>900.00</td><td>270.00</td></tr><tr><td>300.00</td><td>90.00</td></tr><tr><td>350.00</td><td>105.00</td></tr><tr><td>700.00</td><td>210.00</td></tr><tr><td>1500.00</td><td>450.00</td></tr></table>	avg_rent	total_deposit	480.00	144.00	650.00	195.00	500.00	150.00	420.00	126.00	1200.00	360.00	900.00	270.00	300.00	90.00	350.00	105.00	700.00	210.00	1500.00	450.00
avg_rent	total_deposit																						
480.00	144.00																						
650.00	195.00																						
500.00	150.00																						
420.00	126.00																						
1200.00	360.00																						
900.00	270.00																						
300.00	90.00																						
350.00	105.00																						
700.00	210.00																						
1500.00	450.00																						
iii. Monthly payment totals grouped by year & month																							
<pre>SELECT YEAR(p.paid_on) AS year, MONTH(p.paid_on) AS month, SUM(p.amount) AS total_payment FROM payments p GROUP BY year, month;</pre> ‘YEAR(p.paid_on)’ and ‘MONTH(p.paid_on)’ extract from the table ‘payments’. SUM the payment amount into ‘total_payment’ column. GROUP BY year and month to see the unique values calculated.	<table><tr><th>year</th><th>month</th><th>total_payment</th></tr><tr><td>2024</td><td>6</td><td>850.00</td></tr><tr><td>2024</td><td>7</td><td>1120.00</td></tr><tr><td>2024</td><td>8</td><td>1200.00</td></tr><tr><td>2024</td><td>9</td><td>950.00</td></tr><tr><td>2024</td><td>10</td><td>900.00</td></tr></table>	year	month	total_payment	2024	6	850.00	2024	7	1120.00	2024	8	1200.00	2024	9	950.00	2024	10	900.00				
year	month	total_payment																					
2024	6	850.00																					
2024	7	1120.00																					
2024	8	1200.00																					
2024	9	950.00																					
2024	10	900.00																					
iv. Count of OPEN maintenance issues per floor using HAVING COUNT > 2																							
<pre>SELECT r.floor_no, COUNT(m.maint_id) AS maint_issues FROM rooms r LEFT JOIN maintenance m ON r.room_id = m.room_id AND m.status = 'OPEN' GROUP BY r.floor_no HAVING COUNT(m.maint_id) > 2;</pre> COUNT the number of open maintenance issues per floor. LEFT JOIN is used to connect the table of ‘maintenance’ and ‘room’ when ‘m.status’ = ‘OPEN’.	<table><tr><th>floor_no</th><th>maint_issues</th></tr><tr><td>1</td><td>3</td></tr></table>	floor_no	maint_issues	1	3																		
floor_no	maint_issues																						
1	3																						
3. APPLY SQL FUNCTIONS																							

```

SELECT
    CONCAT('Floor ', r.floor_no, ' - ', UPPER(rt.type_name)) AS room_label,
    ROUND(rt.rent, 2) AS rent,
    ROUND(rt.deposit, 2) AS deposit,
    COUNT(DISTINCT CASE WHEN s.status = 'ACTIVE' THEN s.student_id END) AS n_occupant,
    COUNT(DISTINCT CASE WHEN m.status = 'OPEN' THEN m.maint_id END) AS open_issues,
    CASE
        WHEN rt.rent <= 500 THEN 'LOW'
        WHEN rt.rent BETWEEN 501 AND 1000 THEN 'MEDIUM'
        ELSE 'HIGH'
    END AS rent_category
FROM rooms r
JOIN room_types rt ON r.type_id = rt.type_id
LEFT JOIN students s ON r.room_id = s.room_id
LEFT JOIN maintenance m ON r.room_id = m.room_id
GROUP BY r.room_id, r.floor_no, rt.type_id, rt.rent, rt.deposit
ORDER BY r.floor_no;

```

Generate a summary of each room, combining the 'floor_no', 'type_name', ROUND 'rent' value in 2 decimal points, ROUND 'deposit' value in 2 decimal points, COUNT the number of student in each room, COUNT the number of open maintenance issue in each room, and categorized the room class into three classes by the rent value of each room.

LEFT JOIN includes rooms with no students or issues, and GROUP BY to ensure accurate counts per room, while ORDER BY sorts by the floor number.

room_label	rent	deposit	n_occupant	open_issues	rent_category
Floor 1 - SINGLE	350.00	105.00	1	0	LOW
Floor 1 - DOUBLE	500.00	150.00	1	1	LOW
Floor 1 - PREMIUM	900.00	270.00	1	0	MEDIUM
Floor 1 - SINGLE	350.00	105.00	1	1	LOW
Floor 1 - SINGLE	350.00	105.00	1	1	LOW
Floor 2 - FAMILY	1200.00	360.00	1	1	HIGH
Floor 2 - ECONOMY	420.00	126.00	1	0	LOW
Floor 2 - STUDIO	700.00	210.00	0	1	MEDIUM
Floor 3 - DELUXE	650.00	195.00	1	1	MEDIUM
Floor 3 - SUITE	1500.00	450.00	0	0	HIGH
Floor 3 - ACCESSI...	480.00	144.00	0	0	LOW
Floor 3 - SHARED	300.00	90.00	0	0	LOW

MARKING RUBRICS

Task	Criteria	Marks	Total Marks
Task 1 Database Creation & Constraints	Creation of database and all tables with valid keys, datatypes, and constraints (PK, FK, NOT NULL, UNIQUE). Logical order and referential integrity preserved.	3	
	ALTER TABLE (add email UNIQUE) and DROP TABLE demonstration executed successfully and documented.	2	
	Insertion order, valid FK references, and no orphan data.	2	
Task 2 Data Manipulation & Filtering	Five tables populated with ≥ 10 valid records each; varied and meaningful data UPDATE/DELETE executed correctly with logical results.	1	
	Accurate use of WHERE, BETWEEN, LIKE, IN, AND/OR/NOT queries.	3	
	Correct use of aggregate and string functions (COUNT, CONCAT, and more).	2	
Task 3 Reporting & Aggregation	VIEW creation	2	
	Use of GROUP BY, HAVING, ROUND, and CASE with accurate aggregated results.	3	
	Clear labelling, readable outputs, screenshots show executed results.	2	
Total (20)			